

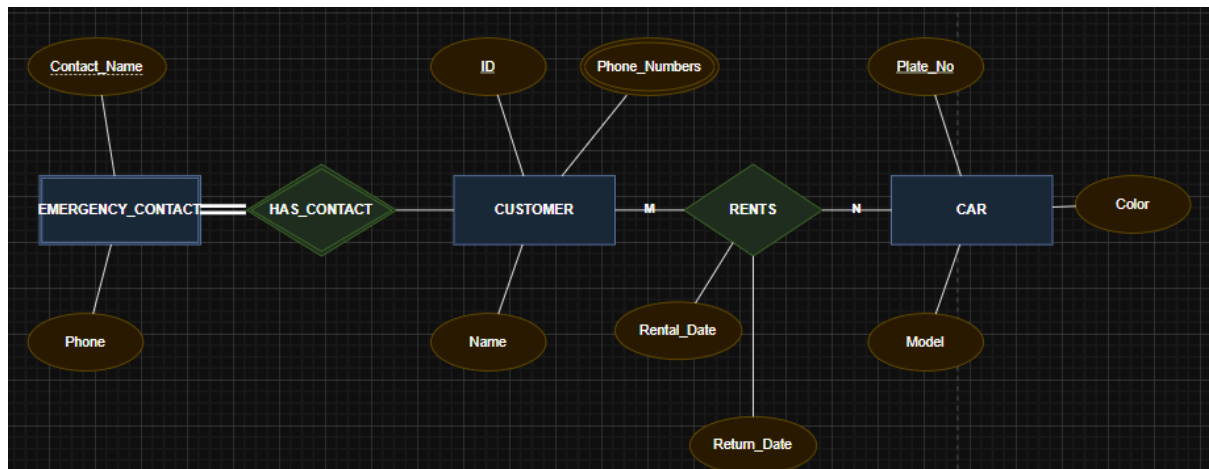
SECTION 1:

Read the following business requirements for a "Car Rental System" and draw the ER Diagram.

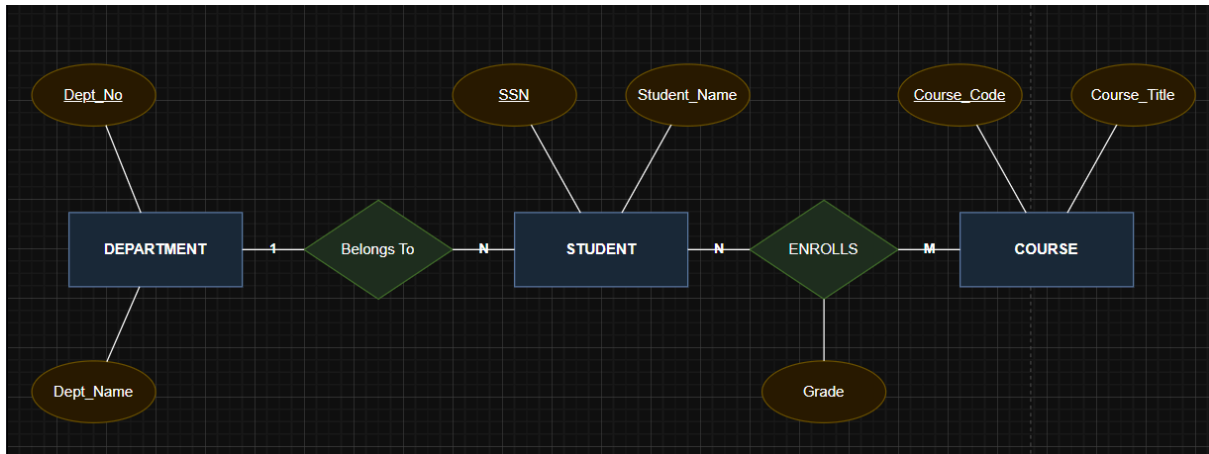
- The system tracks **CUSTOMERS**. Each customer has a unique **ID**, a **Name**, and a multivalued attribute **Phone_Numbers**.
- The system tracks **CARS**. Each car has a unique **Plate_No**, **Model**, and **Color**.
- A Customer can rent many Cars, and a Car can be rented by many Customers over time. This relationship is called **RENTS**, and we need to record the **Rental_Date** and **Return_Date** on this relationship.
- A Customer may optionally have an **EMERGENCY_CONTACT**. The Emergency Contact has a **Contact_Name** and **Phone**. An Emergency Contact cannot exist in the database without being linked to a Customer.
- Not every Customer has an Emergency Contact , but every Emergency Contact **MUST** belong to a Customer .

Your Task:

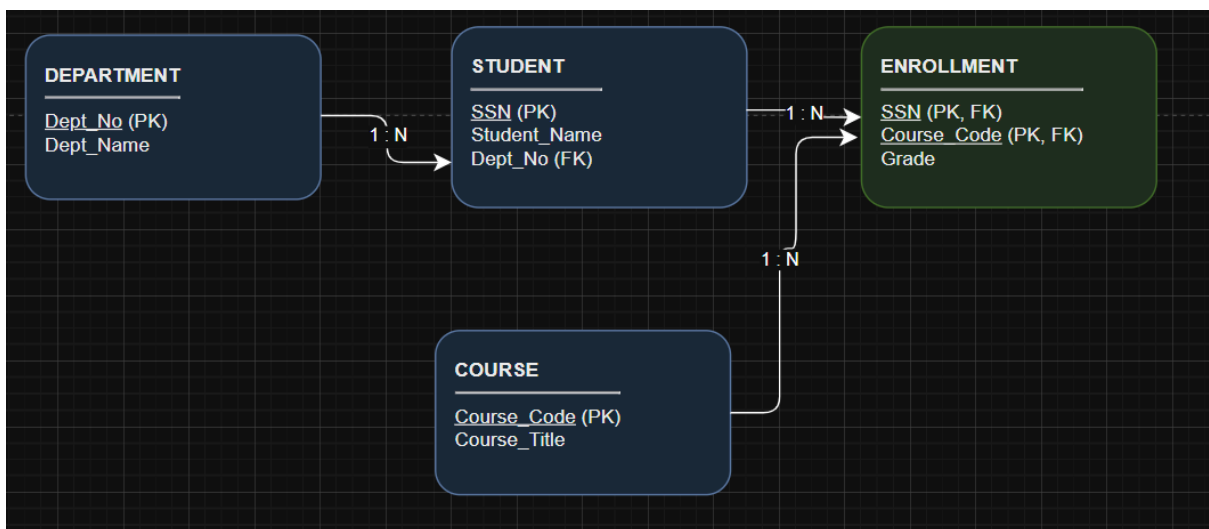
1. convert to ERD



SECTION 2: ERD to Relational Schema Mapping



Your Task: Map this ERD into a Relational Schema



SECTION 3:SQL Queries

Database Schema:

EMPLOYEE (Emp_ID, Name, Salary, Dept_ID) , **DEPARTMENT** (Dept_ID, Dept_Name)
PROJECT (Proj_ID, Proj_Name, Dept_ID) , **WORKS_ON** (Emp_ID, Proj_ID, Hours)

Your Task: Write the SQL **SELECT** statements for the following requests.

1. Retrieve the Employee **Name**, the Project **Proj_Name**, and the **Hours** they worked on it.
2. Retrieve the **Dept_ID** and the total number of employees in each department, but **ONLY** show departments that have **more than 10** employees.
3. Retrieve the **Name** of all employees who work in the '**CS Management Team**' and '**CS Support Team**'. You **MUST** use a nested query with the **IN** operator and **LIKE** (do not use a standard join).

1)

```
SELECT E.Name,P.Proj_Name,W.Hours
FROM EMPLOYEE E, PROJECT P, WORKS_ON W
Where E.Emp_ID = W.Emp_ID AND W.Proj_ID = P.Proj_ID;
```

2)

```
SELECT D.Dept_ID,COUNT(E.Emp_ID)
FROM DEPARTMENT D,EMPLOYEE E
WHERE D.Dept_ID = E.Dept_ID
GROUP BY Dept_ID
HAVING COUNT(E.Emp_ID) > 10;
```

(الحل الي فوق اكثر دقة لانه بيتأكد من وجود الديبارتمنت اول بس مو مطلوب)

(الحل البديل :

```
SELECT Dept_ID, COUNT(Emp_ID)
FROM EMPLOYEE
GROUP BY Dept_ID
HAVING COUNT(Emp_ID) > 10;
```

3)

```
SELECT Name
FROM EMPLOYEE
WHERE Dept_ID IN(
    SELECT Dept_ID
    FROM DEPARTMENT
    WHERE Dept_Name LIKE 'CS%') ;
```

SECTION 4: Relational Algebra

Translate the following simple SQL query into Relational Algebra notation using Sigma and Pi .

SQL: `SELECT Name, Salary FROM EMPLOYEE WHERE Dept_ID = 5;`

$\pi_{\text{Name, Salary}}(\sigma_{\text{Dept_ID} = 5}(\text{EMPLOYEE}))$

SECTION 5: Normalization

You are given the following unnormalized table tracking employee project assignments.

EMP_PROJ (Emp_ID, Proj_ID, Emp_Name, Proj_Name, Hours)

Functional Dependencies (FDs):

- $\text{Emp_ID} \rightarrow \text{Emp_Name}$
- $\text{Proj_ID} \rightarrow \text{Proj_Name}$
- $\{\text{Emp_ID}, \text{Proj_ID}\} \rightarrow \text{Hours}$

Your Task:

1. State why this table violates 2NF.
2. Decompose this table step-by-step into **3NF** relations. Write the final resulting tables and their Primary Keys.

Violation: it's not a 2nf bc hours Emp_Name relies only on Emp_ID and the same for Proj_ID and Proj_Name

3NF :

EMPLOYEE(Emp_ID, Emp_Name)

PROJECT(Proj_ID, Proj_Name)

WORKS(Emp_ID, Proj_ID, Hours)

SECTION 6: Disk Storage

As a DBA, you are analyzing storage performance.

- Your system's **Block Size** is 8192 bytes.
- Your table's **Record Size** is 350 bytes.

Calculate the **Blocking Factor (BFR)**. Show the formula and the final integer result.

the law : $\text{floor}(\text{block size} / \text{record size})$

$$8192 / 350 = 23.4057142857$$

floor the value = 23

SECTION 7: Concepts

1. **(True/False)** In a 1:N relationship mapping, you must create a completely new third table to link the primary keys.
2. **(True/False)** An attribute that depends on only part of a composite primary key is called a Partial Dependency, and removing it is the goal of 2NF.
3. **(True/False)** A Weak Entity has its own independent Primary Key and does not rely on any other table.
4. **(MCQ)** Which SQL clause is used to filter groups *after* an aggregate function (like SUM or COUNT) has been applied?
A) WHERE B) IN
C) **HAVING** D) EXISTS
5. **(MCQ)** You have a table in 2NF. You notice that **Manager_ID** depends on **Dept_ID**, and **Dept_ID** depends on the primary key **Emp_ID**. What rule does this violate, and what normal form fixes it?
A) Partial Dependency (Fix with 2NF) **B) Transitive Dependency (Fix with 3NF)**
C) Multivalued Attribute (Fix with 1NF) D) Entity Integrity (Fix with Primary Key)